

Paso 5 - Decidir | Ejercicios

`if`, `else if`, `else`, `switch`, y la regla de oro: **los textos se comparan con `equals`, nunca con `==`**.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B5_1_ParImpar.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B5.1 - Par o impar

Qué prácticas: un `if` / `else if` / `else` con un caso especial, y el operador `%` para probar paridad.

Pide un número entero y di si es par o impar. Después agrégale el caso del cero, que no es ni lo uno ni lo otro para este ejercicio:

```
Numero: 0
El cero no cuenta
```

Pista: `numero % 2 == 0`.

B5.2 - El tramo de edad

Qué prácticas: encadenar `else if` para cubrir rangos, y por qué el orden de las condiciones importa.

Pide la edad y responde:

Edad	Respuesta
menos de 0 o más de 120	Esa edad no existe
0 a 17	Menor de edad

Edad	Respuesta
18 a 64	Adulto
65 o más	Adulto mayor

Edad: -5
Esa edad no existe

Edad: 15
Menor de edad

Edad: 80
Adulto mayor

Fíjate en el orden de los `else if`: si pones primero el `18 a 64`, el `-5` nunca llega a la validación. Pruébalo al revés para verlo.

B5.3 - La clave

Qué practicas: por qué comparar textos con `==` casi siempre está mal, y cuándo usar `equals` en vez de `equalsIgnoreCase`.

Pide un usuario y una clave. El acceso es correcto si el usuario es `alumno` (sin importar mayúsculas) y la clave es exactamente `duoc2026`.

```
Usuario: ALUMNO
Clave: duoc2026
Bienvenido, ALUMNO
```

1. Escríbelo primero con `==` para comparar la clave. Compila sin aviso y **siempre** falla: compruébalo.
2. Arréglalo con `equals`.
3. Para el usuario usa `equalsIgnoreCase`, y explica en un comentario por qué para la clave **no** conviene usarlo.

B5.4 - La calculadora con `switch`

Qué practicas: un `switch` con `default`, y validar un caso especial (dividir por cero) antes de que ocurra.

Pide dos números y un número de operación (1 sumar, 2 restar, 3 multiplicar, 4 dividir). Usa `switch` para elegir.

```
Primer numero: 8
Segundo numero: 0
1) Sumar  2) Restar  3) Multiplicar  4) Dividir
Operacion: 4
No se puede dividir por cero
```

Requisitos:

- Si la operación no es 1, 2, 3 ni 4: `Esa operacion no existe` (el `default`).
- Si es dividir y el segundo número es 0: el mensaje de arriba, **sin** que el programa se caiga.

Prueba qué pasa si divides `8 / 0` con enteros y sin el `if`: eso es una `ArithmeticException`, y vuelve en la demo 10.

B5.5 - El mayor de tres

Qué practicas: resolver el mismo problema con `if` anidados y con `Math.max`, para comparar cuál se lee mejor.

Pide tres números y di cuál es el mayor. Hazlo de **dos** formas:

1. Con `if` anidados, sin usar `Math`.
2. Con `Math.max(Math.max(a, b), c)`.

```
Primer numero: 3
Segundo numero: 9
Tercer numero: 5
Con if:          9
Con Math.max:    9
```

¿Cuál de las dos se lee mejor? ¿Y si fueran diez números?

B5.6 - Desafío: la nota en palabras

Qué practicas: encadenar rangos con `if`, y por qué un `switch` no puede reemplazarlos.

Pide una nota de 1.0 a 7.0 y muestra su categoría:

Nota	Texto
6.5 a 7.0	Excelente
5.5 a 6.4	Muy buena
4.0 a 5.4	Suficiente
1.0 a 3.9	Insuficiente
fuera de rango	Esa nota no existe

Nota (1.0 a 7.0): 6.8
Excelente

Nota (1.0 a 7.0): 7.5
Esa nota no existe

Después responde: ¿se puede hacer esto con un `switch`? ¿Por qué?

Cuando termines, sigue con el [paso 6](#).